Practical SAT Solving

Lecture 1: Introduction

Prof. Dr. Carsten Sinz

Hochschule Karlsruhe, University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik
Lehrgebiet Software-Entwicklung



Organization

- Lectures
  - 2 ECTS / 2 SWS
  - Every Monday at 11:30
  - Please enroll to the lecture in ILIAS
- Exercises
  - 2 ECTS / 1 SWS
  - Every second Monday (starting October 5) at 14:00

Exam

- Exam
  - Written exam
- Exercises
  - You will need at least 50% of the exercise points to get the ECTS points for the exercises

Contact

- Carsten Sinz
  - email: carsten.sinz@h-ka.de
  - consultation hours (by appointment): Wednesday 13:00-14:00

Goals of this Lecture

- How to use a SAT solver efficiently
  - How to encode stuff into CNF
- How do SAT solvers work
  - Algorithms
- How to make a SAT solver
  - Implementation techniques

Propositional Logic

- A *Boolean variable* (x) is a variable with two possible values: *True* and *False*.
- A *literal* is a Boolean variable x (positive literal) or its negation $\overline{x}$ (negative literal).
- A *clause* is a disjunction (or $= \vee$) of literals.
- A *CNF*[1] *formula* is a conjunction (and $= \wedge$) of clauses.

Example

$F = (\overline{x_1} \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3) \wedge (x_1)$
$Vars(F) = \{x_1, x_2, x_3\}$
$Lits(F) = \{x_1, \overline{x_1}, x_2, \overline{x_2}, x_3\}$
$Cls(F) = \{(\overline{x_1} \vee x_2), (\overline{x_1} \vee \overline{x_2} \vee x_3), (x_1)\}$

[1] Conjunctive Normal Form

Satisfiability

- A *truth assignment* ϕ assigns a truth value (True or False) to each Boolean variable x, i.e., $\phi(x) = \mathit{True}$ or $\phi(x) = \mathit{False}$.
- We say that ϕ satisfies
  - a positive literal x if $\phi(x) = \mathit{True}$
  - a negative literal $\overline{x}$ if $\phi(x) = \mathit{False}$
  - a clause if it satisfies at least one of its literals
  - a CNF formula if it satisfies all of its clauses

Satisfiability

- A *truth assignment* ϕ assigns a truth value (True or False) to each Boolean variable x, i.e., $\phi(x) = \mathit{True}$ or $\phi(x) = \mathit{False}$.
- We say that ϕ satisfies
  - a positive literal x if $\phi(x) = \mathit{True}$
  - a negative literal $\overline{x}$ if $\phi(x) = \mathit{False}$
  - a clause if it satisfies at least one of its literals
  - a CNF formula if it satisfies all of its clauses
- If ϕ satisfies a CNF F then we call ϕ a *satisfying assignment* of F.
- A formula F is *satisfiable* if there is ϕ that satisfies F.
- The *Satisfiability Problem* is to determine whether a given formula is satisfiable. If so, we would also like to see a satisfying assignment.

Satisfiability - Examples

Satisfiable Formulas

(x_1)

$(x_2 \vee x_8 \vee \overline{x_3})$

$(\overline{x_1} \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3) \wedge (x_1)$

Unsatisfiable Formulas

$(x_1) \wedge (\overline{x_1})$

$(x_1) \wedge (\overline{x_1}) \wedge (x_2 \vee x_8 \vee \overline{x_3})$

$(x_1 \vee x_2) \wedge (\overline{x_1} \vee x_2) \wedge (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee \overline{x_2})$

Satisfiability - A Practical Example

Scheduling a meeting consider the following constraints

- Adam can only meet on Monday or Wednesday
- Bridget cannot meet on Wednesday
- Charles cannot meet on Friday
- Darren can only meet on Thursday or Friday

Expressed as SAT

$$F = (x_1 \vee x_3) \wedge (\overline{x_3}) \wedge (\overline{x_5}) \wedge (x_4 \vee x_5) \wedge \text{AtMostOne}(x_1, x_2, x_3, x_4, x_5)$$

Satisfiability - A Practical Example

Scheduling a meeting consider the following constraints

- Adam can only meet on Monday or Wednesday
- Bridget cannot meet on Wednesday
- Charles cannot meet on Friday
- Darren can only meet on Thursday or Friday

Expressed as SAT

$$F = (x_1 \vee x_3) \wedge (\overline{x_3}) \wedge (\overline{x_5}) \wedge (x_4 \vee x_5)$$
$$\wedge(\overline{x_1} \vee \overline{x_2}) \wedge (\overline{x_1} \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_4}) \wedge (\overline{x_1} \vee \overline{x_5})$$
$$\wedge(\overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_2} \vee \overline{x_4}) \wedge (\overline{x_2} \vee \overline{x_5})$$
$$\wedge(\overline{x_3} \vee \overline{x_4}) \wedge (\overline{x_3} \vee \overline{x_5})$$
$$\wedge(\overline{x_4} \vee \overline{x_5})$$

Satisfiability - A Practical Example

Scheduling a meeting consider the following constraints

- Adam can only meet on Monday or Wednesday
- Bridget cannot meet on Wednesday
- Charles cannot meet on Friday
- Darren can only meet on Thursday or Friday

Expressed as SAT

$F = (x_1 \vee x_3) \wedge (\overline{x_3}) \wedge (\overline{x_5}) \wedge (x_4 \vee x_5) \wedge \text{AtMostOne}(x_1, x_2, x_3, x_4, x_5)$

Solution: Unsatisfiable, i.e., it is impossible to schedule a meeting with these constraints

Satisfiability - Hardness

Satisfiability is NP-Complete [Coo71], proof idea:

- SAT is in NP – easy, checking a solution can be done in linear time
- SAT is NP-hard – encode the run of a non-deterministic Turing machine on an input to a CNF formula.

Consequences:

- We don't have a polynomial algorithm for SAT (yet) :(
- If $P \neq NP$ then we won't have a polynomial algorithm :'(
- All the known complete algorithms have exponential runtime in the **worst case**.

Hardness, try it yourself:
`http://www.cs.utexas.edu/~marijn/game/`

Satisfiability - History

- 1960 The first SAT solving algorithm DP (Davis, Putnam) [DP60]
- 1962 An improved version of DP – DPLL [DLL62]
- 1971 SAT was the first NP-Complete problem [Coo71]
- 1992 Local Search SAT solving [SLM+92]
- 1992 The First International SAT Competition, followed by 1993, 1996, since 2002 every year
- 1996 Conflict Driven Clause Learning [MSS99]
- 1996 The First International SAT Conference (Workshop), followed by 1998, since 2000 every year

Early 90's: 100 variables, 200 clauses, Today: 1,000,000 variables and 5,000,000 clauses.

SAT Conference 2015



Applications of SAT Solving

- Hardware Model Checking
  - All major hardware companies (Intel, ...) use SAT solver to verify their chip designs
- Software Verification
  - SAT solver based SMT solvers are used to verify Microsoft software products
  - Embedded software in cars, airplanes, refrigerators, ...
  - Unix utilities
- Automated Planning and Scheduling in Artificial Intelligence
  - Still one of the best approaches for optimal planning
- Number Theoretic Problems (Pythagorean Triples)
- Solving other NP-hard problems (coloring, clique, ...)

SAT Solving in the news

SPIEGEL ONLINE DER SPIEGEL SPIEGEL TV Anmelden

WISSENSCHAFT Schlagzeilen | Wetter | DAX 12.060,78 | TV-Programm | Abo

Nachrichten > Wissenschaft > Mensch > Mathematik > Der längste Mathe-Beweis der Welt umfasst 200 Terabyte

Zahlenrätsel

Der längste Mathe-Beweis der Welt

Drei Mathematiker haben ein Zahlenrätsel geknackt - mithilfe eines Supercomputers. Der Beweis umfasst 200 Terabyte. Sie wollen wissen, worum es geht? Okay, versuchen wir es.



Von Holger Dambeck



DPA

Supercomputer als Mathematiker

Pythagorean Triples

Problem Definition

Is it possible to assign to each integer $1, 2, \ldots, n$ one of two colors such that if $a^2 + b^2 = c^2$ then a, b and c do not all have the same color.

Pythagorean Triples

Problem Definition

Is it possible to assign to each integer $1, 2, \ldots, n$ one of two colors such that if $a^2 + b^2 = c^2$ then a, b and c do not all have the same color.

- Solution: Nope
- for $n = 7825$ it is not possible
- proof obtained by a SAT solver has 200 Terrabytes – the largest Math proof ever

Pythagorean Triples

Problem Definition

Is it possible to assign to each integer $1, 2, \ldots, n$ one of two colors such that if $a^2 + b^2 = c^2$ then a, b and c do not all have the same color.

- Solution: Nope
- for $n = 7825$ it is not possible
- proof obtained by a SAT solver has 200 Terrabytes – the largest Math proof ever

How to encode this?

- for each integer i we have a Boolean variable x_i, $x_i = 1$ if color of i is 1, $x_i = 0$ otherwise.
- for each a, b, c such that $a^2 + b^2 = c^2$ we have two clauses: $(x_a \vee x_b \vee x_c)$ and $(\overline{x_a} \vee \overline{x_b} \vee \overline{x_c})$

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011?

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011? No, the 1s at x_2, x_5, x_8 are equally spaced.

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011? No, the 1s at x_2, x_5, x_8 are equally spaced.
- 6 Solutions:
  00110011, 01011010, 01100110, 10011001, 10100101, 11001100.

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011? No, the 1s at x_2, x_5, x_8 are equally spaced.
- 6 Solutions:
  00110011, 01011010, 01100110, 10011001, 10100101, 11001100.
- Extending the problem to 9 digits, no solutions remain. How can we show this with a SAT solver?

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011? No, the 1s at x_2, x_5, x_8 are equally spaced.
- 6 Solutions:
  00110011, 01011010, 01100110, 10011001, 10100101, 11001100.
- Extending the problem to 9 digits, no solutions remain. How can we show this with a SAT solver?
- Encode what's forbidden: $x_2 x_5 x_8 \neq 111$ is the same as $(\overline{x_2} \vee \overline{x_5} \vee \overline{x_8})$.

Arithmetic Progressions

Find a binary sequence $x_1, \ldots, x_8$ that has no three equally spaced 0s and no three equally spaced 1s.

- What about 01001011? No, the 1s at x_2, x_5, x_8 are equally spaced.
- 6 Solutions:
  00110011, 01011010, 01100110, 10011001, 10100101, 11001100.
- Extending the problem to 9 digits, no solutions remain. How can we show this with a SAT solver?
- Encode what's forbidden: $x_2 x_5 x_8 \neq 111$ is the same as $(\overline{x_2} \vee \overline{x_5} \vee \overline{x_8})$.
- Writing, e.g., $\bar{2}\bar{5}\bar{8}$ for the clause $(\overline{x_2} \vee \overline{x_5} \vee \overline{x_8})$, we arrive at 32 clauses for the 9 digit sequence:
  $123, 234, \ldots, 789, 135, 246, \ldots, 579, 147, 258, 369, 159,$
  $\bar{1}\bar{2}\bar{3}, \bar{2}\bar{3}\bar{4}, \ldots, \bar{7}\bar{8}\bar{9}, \bar{1}\bar{3}\bar{5}, \bar{2}\bar{4}\bar{6}, \ldots, \bar{5}\bar{7}\bar{9}, \bar{1}\bar{4}\bar{7}, \bar{2}\bar{5}\bar{8}, \bar{3}\bar{6}\bar{9}, \bar{1}\bar{5}\bar{9}.$

Background: Van der Waerden Numbers

Theorem (van der Waerden)

If n is sufficiently large, every sequence $x_1, \ldots, x_n$ of numbers $0 \leq x_i < r$ contains a number that occurs at least k times equally spaced.

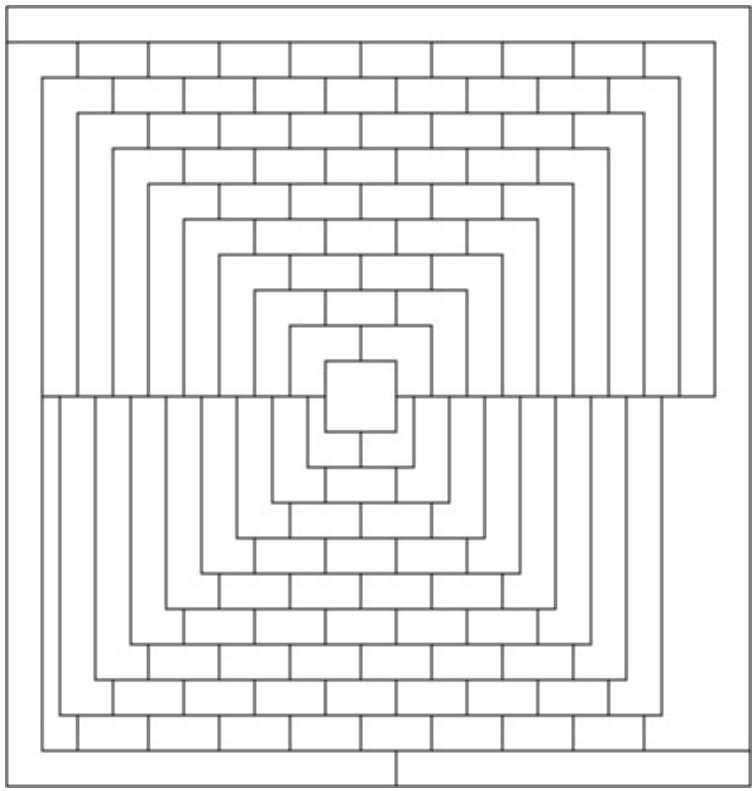
- The smallest such number is the van der Waerden number $W(r, k)$.
- We have seen that $W(2, 3) = 9$.
- For larger r, k the numbers are only partially known.
- E.g., $W(2, 6) = 1132$ was shown in 2008 by Kouril and Paul [KP08] (with the help of a SAT solver!), but $W(2, 7)$ is yet unknown.
- $2^{2^{r^{2^{2^{k+9}}}}}$ is an upper bound for $W(r, k)$ (shown in 2001 by Gowers [Gow01]).

Graph Coloring

- McGregor graph (of order 10): planar, 110 nodes.
- Claim: Cannot be colored with less than 5 colors. (Scientific American, 1975, Martin Gardner's column "Mathematical Games")

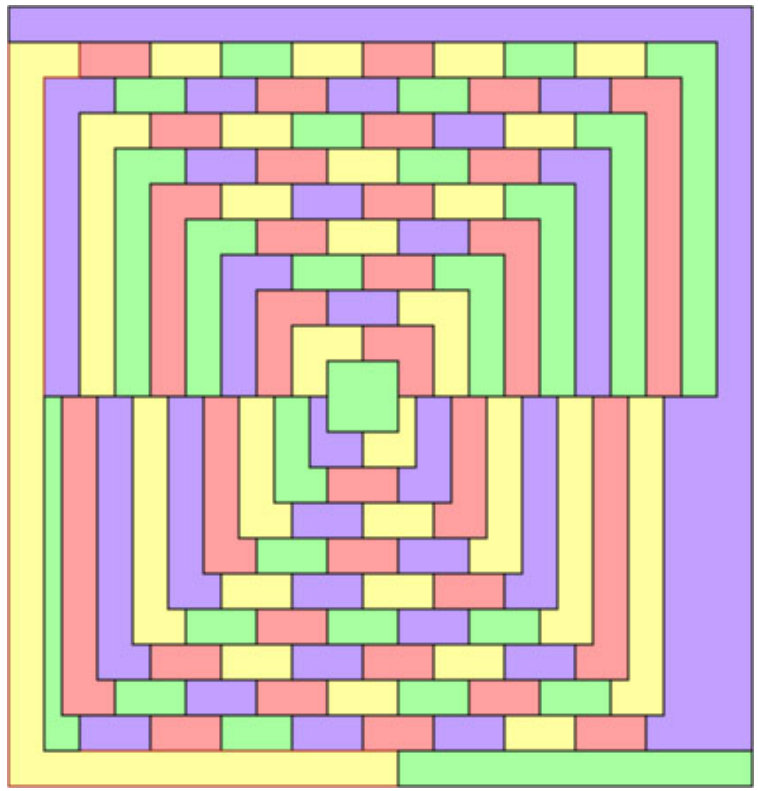
Graph Coloring

- McGregor graph (of order 10): planar, 110 nodes.
- Claim: Cannot be colored with less than 5 colors. (Scientific American, 1975, Martin Gardner's column "Mathematical Games")



Graph Coloring

- McGregor graph (of order 10): planar, 110 nodes.
- Claim: Cannot be colored with less than 5 colors. (Scientific American, 1975, Martin Gardner's column "Mathematical Games")



Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.
- Clauses?

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.
- Clauses?

  $(v_1 \vee \cdots \vee v_k)$ for $v \in V$ ("every node gets a color")

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.
- Clauses?

  $(v_1 \vee \cdots \vee v_k)$ for $v \in V$ ("every node gets a color")

  $(\overline{u_j} \vee \overline{v_j})$ for $u \text{—} v, 1 \leq j \leq k$ ("adjacent nodes have diff. colors")

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.
- Clauses?

  $(v_1 \vee \cdots \vee v_k)$ for $v \in V$ ("every node gets a color")

  $(\overline{u_j} \vee \overline{v_j})$ for $u \text{—} v, 1 \leq j \leq k$ ("adjacent nodes have diff. colors")
- What about multiple colors for a node?

Graph Coloring: Encoding in SAT

Definition: Graph coloring

Given an undirected graph $G = (V, E)$, a graph coloring (proper vertex coloring) assigns a color to each node, such that all adjacent nodes have a different color. A graph coloring using at most k colors is called a k-coloring. The Graph Coloring Problem asks whether a k-coloring for G exists.

- SAT encoding: use $k \cdot |V|$ Boolean variables v_j for $v \in V$, $1 \leq j \leq k$, where v_j is true, if node v gets color j.
- Clauses?

  $(v_1 \vee \cdots \vee v_k)$ for $v \in V$ ("every node gets a color")

  $(\overline{u_j} \vee \overline{v_j})$ for $u \text{—} v, 1 \leq j \leq k$ ("adjacent nodes have diff. colors")
- What about multiple colors for a node? $\rightarrow$ At-most-one constraints

Graph Coloring: Example

- $V = \{u, v, w, x, y\}$
- Colors: red (=1), green (=2), blue (=3)
- Clauses:

  "every node gets a color"

  $(u_1 \vee u_2 \vee u_3)$

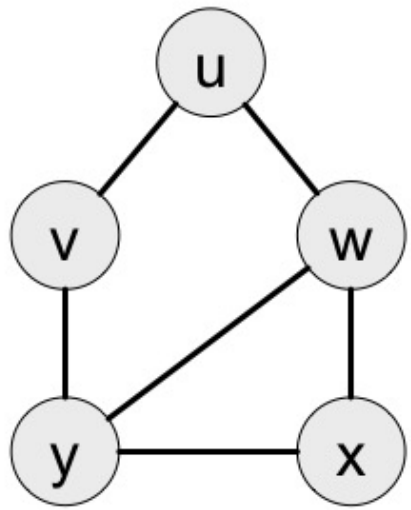
  $\vdots$

  $(y_1 \vee y_2 \vee y_3)$

  "adjacent nodes have different colors" $(\overline{u_1} \vee \overline{v_1}) \wedge \cdots \wedge (\overline{u_3} \vee \overline{v_3})$

  $\vdots$

  $(\overline{x_1} \vee \overline{y_1}) \wedge \cdots \wedge (\overline{x_3} \vee \overline{y_3})$



Graph Coloring: Example

- $V = \{u, v, w, x, y\}$
- Colors: red (=1), green (=2), blue (=3)
- Clauses:

  "every node gets a color"

  $(u_1 \vee u_2 \vee u_3)$

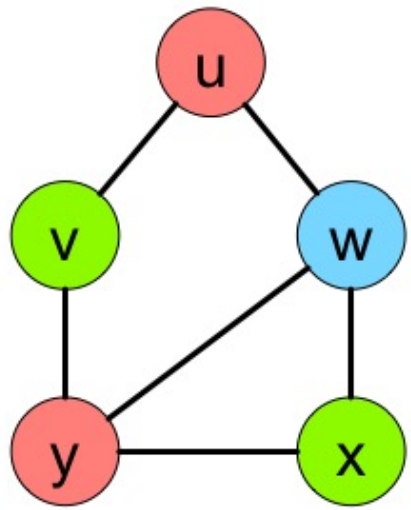
  $\vdots$

  $(y_1 \vee y_2 \vee y_3)$

  "adjacent nodes have different colors" $(\overline{u_1} \vee \overline{v_1}) \wedge \cdots \wedge (\overline{u_3} \vee \overline{v_3})$

  $\vdots$

  $(\overline{x_1} \vee \overline{y_1}) \wedge \cdots \wedge (\overline{x_3} \vee \overline{y_3})$



Using a SAT Solver – Input

- SAT solvers are command line applications that take 1 argument – a text file with a formula which looks like (DIMACS format):

INPUT (myformula.txt)

```
c comments, ignored by solver
p cnf 7 22
1 -2 7 0
...
-7 -3 -2 0
```

- lines starting "c" are comments and are ignored by the SAT solver
- a line starting with "p cnf" is the problem definition line containing the number of variables and clauses.
- the rest of the lines represent clauses, literals are integers (starting with variable 1), clauses are terminated by a zero.

Using a SAT Solver – Output

- calling `./mysolver myformula.txt` would print on the screen something like

OUTPUT (./solver myformula.txt)

```
c comments, usually some stastitics about the solving
s SATISFIABLE
v 1 2 -3 -4
v 5 -6 -7 0
```

- the solution line (starting with "s") can also contain UNSATISFIABLE and UNKNOWN
- the truth values of variables are printed in lines starting with "v", the last value is followed by a "0"

Incremental SAT Solving

- We often need to solve a sequence of similar SAT instances
  - for example planning as SAT, Sokoban, bounded model checking
  - the instances share most of the clauses with their neighbors
- Can we solve these sequences of instances more efficiently?

Incremental SAT Solving

- We often need to solve a sequence of similar SAT instances
  - for example planning as SAT, Sokoban, bounded model checking
  - the instances share most of the clauses with their neighbors
- Can we solve these sequences of instances more efficiently?
- What is incremental SAT solving?
  - Clauses can be added to and removed from the SAT solver
- Why not call the solver with the new formula every time?

Incremental SAT Solving

- We often need to solve a sequence of similar SAT instances
  - for example planning as SAT, Sokoban, bounded model checking
  - the instances share most of the clauses with their neighbors
- Can we solve these sequences of instances more efficiently?
- What is incremental SAT solving?
  - Clauses can be added to and removed from the SAT solver
- Why not call the solver with the new formula every time?
  - The solver can remember learned clauses and other stuff (variable scores required for heuristics)
  - (de)initialization overheads removed

Incremental Library Track – IPASIR



- Previously each SAT solver had a different incremental interface
- For the 2015 SAT Race a unified interface was defined – IPASIR
- IPASIR = Re-entrant Incremental Satisfiability Application Program Interface (acronym reversed)
- Currently around 10 SAT solvers are IPASIR compatible

IPASIR Overview

- Based on Lingeling incremental interface
- Clauses are added one literal at a time
  - To add $(x_1 \vee \overline{x_4})$ call `add(1); add(-4); add(0);`

IPASIR Overview

- Based on Lingeling incremental interface
- Clauses are added one literal at a time
  - To add $(x_1 \vee \overline{x_4})$ call `add(1); add(-4); add(0);`
- You can call a SAT solver with a set of assumptions
  - Assumptions are basically temporary unit clauses
  - Assumptions are cleared after each `"solve"` call

IPASIR Overview

- Based on Lingeling incremental interface
- Clauses are added one literal at a time
  - To add $(x_1 \vee \overline{x_4})$ call `add(1); add(-4); add(0);`
- You can call a SAT solver with a set of assumptions
  - Assumptions are basically temporary unit clauses
  - Assumptions are cleared after each `"solve"` call
- Clause removal is done via activation literals and assumptions
  - You must know ahead which clauses you will maybe want to remove
  - Add the clause with an additional fresh variable (activation literal)
  - example: instead of $(x_1 \vee x_2)$ add $(x_1 \vee x_2 \vee a_1)$
  - solve with with assumption $\overline{a_1}$ to enforce $(x_1 \vee x_2)$
  - drop the assumption $\overline{a_1}$ to drop $(x_1 \vee x_2)$

Ipasir Interface

ipasir.h

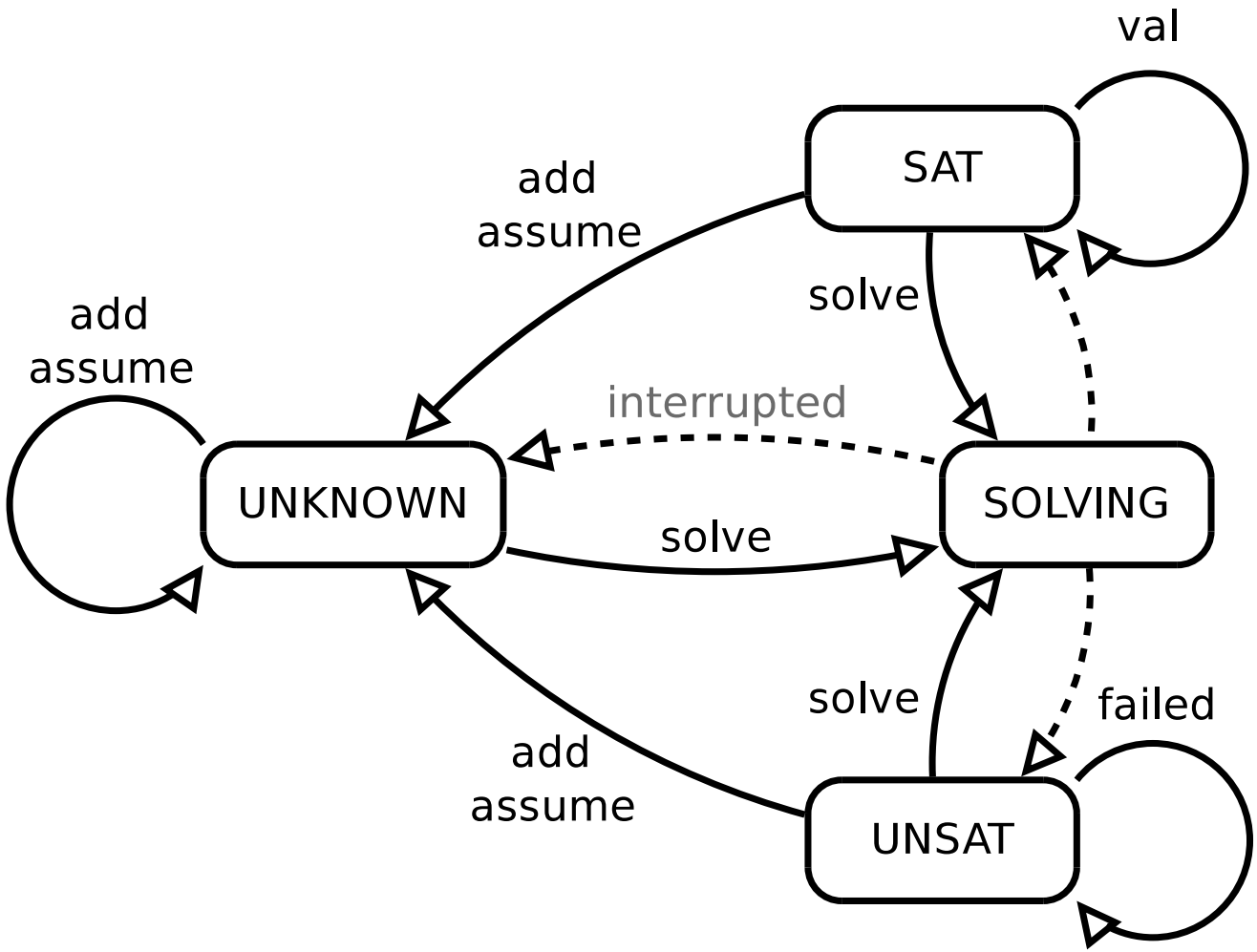
```
const char* ipasir_signature();
void* ipasir_init();
void ipasir_release(void* solver);
void ipasir_set_terminate(void* solver, void* state,
     int (*terminate)(void* state));
void ipasir_set_learn (void * solver, void * state,
     int max_length, void (*learn)(void * state, int * clause));
void ipasir_add(void* solver, int lit_or_zero);
void ipasir_assume(void* solver, int lit);
int ipasir_solve(void* solver);
int ipasir_val(void* solver, int lit);
int ipasir_failed(void* solver, int lit);
```

For more details and examples of usage see `https://github.com/biotomas/ipasir`.

IPASIR Functions

- signature – return the name and version of the solver
- init – initialize the solver, the pointer it returns is used for the rest of the functions
- add – add clauses, one literal at a time
- assume – add an assumption, the assumptions are cleared after a `"solve"` call
- solve – solve the formula, return SAT, UNSAT or INTERRUPTED
- val – return the truth value of a variable (if solve returned SAT)
- failed – returns true if the given assumption was required for the unsatisfiability of the formula (if solver returned UNSAT)

IPASIR Solver States



Example – Essential Variables

- For a satisfiable formula F, a variable x is *essential* if and only x it has to be assigned (True or False) in each satisfying partial assignment of F.
- Example: In $(x \vee y \vee z) \wedge (\overline{x})$ only x is essential.
- Task: find all the essential variables of a given formula
- How to do it:
  - use *Dual Rail Encoding* – for each variable x add two new variables x_P and x_N, replace each positive (negative) occurrence of x with x_P (x_N), add a clause $(\overline{x_P} \vee \overline{x_N})$ (meaning x cannot be both true and false).
  - for each variable x solve the formula with the assumptions $\overline{x_P}$ and $\overline{x_N}$. If the formula is UNSAT then x is essential.

Example – Essential Variables – code

```
1  int pdr(int var) { return 2*var; }
2  int ndr(int var) { return 2*var - 1; }
3  int  dr(int lit) { return lit > 0 ? pdr(lit) : ndr(-lit); }
4
5  void Essentials(Formula f) {
6    void* s = ipasir_init();
7    for (int c = 0; c < f.clauses; c++) {
8      for (int k = 0; k < f.clause[c].size; k++) {
9        ipasir_add(s, dr(f.clause[c].lit[k]));
10     }
11     ipasir_add(s, 0);
12   }
13   for (int v = 1; v <= f.variables; v++) {
14     ipasir_add(s, -pdr(v));
15     ipasir_add(s, -ndr(v));
16     ipasir_add(s, 0);
17   }
18   for (int v = 1; v <= f.variables; v++) {
19     ipasir_assume(s, -pdr(v));
20     ipasir_assume(s, -ndr(v));
21     if (ipasir_solve(s) == 20) {
22       printf("%d is Essential\n", v);
23     } else {
24       printf("%d is not Essential\n", v);
25     }
26   }
27   ipasir_release(s);
28 }
```

References I


Stephen A Cook.
The complexity of theorem-proving procedures.
In *Proceedings of the third annual ACM symposium on Theory of computing*, pages 151–158. ACM, 1971.


Martin Davis, George Logemann, and Donald Loveland.
A machine program for theorem-proving.
Commun. ACM, 5(7):394–397, July 1962.


Martin Davis and Hilary Putnam.
A computing procedure for quantification theory.
J. ACM, 7(3):201–215, July 1960.


W.T. Gowers.
A new proof of Szemerédi's theorem.
Geometric & Functional Analysis GAFA, 11(3):465–588, 2001.

References II


Michal Kouril and Jerome L. Paul.
The van der Waerden Number W(2, 6) is 1132.
Experimental Mathematics, 17(1):53–61, 2008.


João P Marques-Silva and Karem A Sakallah.
Grasp: A search algorithm for propositional satisfiability.
Computers, IEEE Transactions on, 48(5):506–521, 1999.


Bart Selman, Hector J Levesque, David G Mitchell, et al.
A new method for solving hard satisfiability problems.
In *AAAI*, volume 92, pages 440–446, 1992.